

ERPM — Avancement Phase 2 : `erpm_long`, `inertia_groups` et nouvelles transitions MH

Jérémie Chichignoud (CUB'iTECH)

13 janvier 2026

Table des matières

1	Contexte et périmètre	2
1.1	Contexte	2
1.2	Choix des livrables pour la Phase 2	2
1.3	Structure du document	2
2	Livrables sélectionnés (objectifs et validation)	2
2.1	<code>erpm_long</code>	2
2.2	Effet inertiel : <code>inertia_groups</code>	3
2.3	Transitions Metropolis–Hastings et proposal <code>ErpmMix</code>	3
3	Synthèse étape par étape	6
1	ERPM long	6
2	<code>inertia_groups</code> : persistance stricte de groupes entiers	7

État d'avancement : livrables sélectionnés (Phase 2)

1 Contexte et périmètre

1.1 Contexte

La phase 1 est terminée et a conduit au développement de l'interface `erpm`. Cette interface prend en entrée une partition ou un réseau biparti, construit l'objet `network` correspondant (acteurs-groupes), traduit une formule ERPM en appel `ergm()`, et prend en charge la préparation des entrées, la construction de l'appel `ergm()`, ainsi que l'exécution ou le retour contrôlé de cet appel.

1.2 Choix des livrables pour la Phase 2

Les livrables retenus pour cette phase sont :

- implémentation de `erpm_long`, un outil permettant de construire et d'ajuster des ERGM successifs sur des partitions, en intégrant des covariables d'historique afin d'approximer un modèle longitudinal sur les partitions ;
- implémentation d'un unique effet inertiel, `inertia_groups`, mesurant la persistance des groupes entre partitions successives ;
- implémentation de nouvelles transitions Metropolis-Hastings (swap, merge, split), regroupées dans un proposal mixte `ErpmMix` et utilisable via le wrapper `erpm()`.

1.3 Structure du document

Ce document d'avancement présente :

- les objectifs et critères de validation associés aux livrables sélectionnés ;
- un tableau de suivi synthétique (étapes et sous-étapes) ;
- une synthèse étape par étape des travaux réalisés et restants.

2 Livrables sélectionnés (objectifs et validation)

2.1 `erpm_long`

Objectif.

Fournir un outil permettant d'enchaîner des ERPM successives, en injectant des informations issues des partitions passées sous forme d'attribut de réseau, afin d'approcher un modèle ERPM longitudinal sans implémenter un TERGM complet.

Validation.

- Dry-run statique : génération d'une succession de call `ergm()` valides, exclusivement avec des effets statiques, sans exécution.
- Dry-run longitudinal : génération d'une succession de call `ergm()` valides, avec des effets statiques et inertiel, sans exécution. Les réseaux doivent contenir les covariables d'historiques calculées en fonction de l'effet inertiel voulu.
- Fit statique : génération d'une succession de fits `ergm()` valides, exclusivement avec des effets statiques. C'est l'équivalent d'appel successifs à `erpm` sur le même set de partitions.
- Fit longitudinal : génération d'une succession de fits `ergm()` valides, avec des effets statiques et inertiel, utilisant des covariables dérivées des partitions antérieures.

2.2 Effet inertiel : `inertia_groups`

Objectif.

Introduire un terme inertiel qui compte, à un temps t , les groupes dont la composition (ensemble exact d'acteurs) coïncide avec celle d'au moins un groupe observé dans une ou plusieurs partitions passées, sur une fenêtre temporelle donnée.

Validation.

- `InitErgmTerm` : parsing des arguments, construction des `inputs`, nommage des coefficients, compatibilité bipartite.
- Change-stat C : calcul correct de la variation 1-toggle.
- Tests sur plusieurs cas d'usage : combinaison avec des effets statiques, avec plusieurs profondeurs et plusieurs tailles de groupe.

2.3 Transitions Metropolis–Hastings et proposal `ErpmMix`

Objectif.

Étendre l'exploration MCMC via de nouveaux mouvements compatibles multi-toggle : `swap`, `merge`, `split`, chacun étant encodé comme un multi-toggle unique au sens de `ergm`. Ces mouvements sont sélectionnés aléatoirement à chaque itération au sein d'un proposal mixte `ErpmMix`, avec des probabilités contrôlées par l'utilisateur.

Étapes de développement.

- Mise au format multi-toggle des change-stats ERPM (`ntoggles > 1`).
- Fonctions C « steps » : `ErpmToggleStep`, `ErpmSwapStep`, `ErpmMergeStep`, `ErpmSplitStep`.
- Proposal C : `MH_ErpmMix` (init + proposition).
- Interface R : `InitErgmProposal.ErpmMix`.
- Wrapper : ajout des arguments d'entrée `mh_moves` / `mh_weights` à `erpm()` et traduction en `control.ergm()`.

Validation.

- Multi-toggle : tous les termes ERPM restent corrects en multi-toggles.
- Steps : chaque step produit un multi-toggle valide et un `logratio` correct.
- `ErpmMix` : MCMC stables, proportions empiriques de transitions cohérentes avec les poids.
- R + wrapper : l'utilisateur peut activer `ErpmMix` via `erpm(..., mh_moves=..., mh_weights=...)`.

Tableau de suivi

Étape	Sous-étape	Description	Validation	Remarque	Check
erpm_long		Implémentation de <code>erpm_long</code> , outil permettant de construire et d'ajuster des ERGM successifs sur des partitions, en intégrant des covariables d'historique dérivées des partitions précédentes.	Pour plusieurs configurations (nombre de partitions, valeurs de lag et combinaisons d'effets), les modèles <code>ergm()</code> générés sont bien définis et exploitables pour l'estimation.	Ajustements statiques validés. Partie longitudinale en cours de consolidation.	△
↔	dry-run	Construction complète des appels <code>ergm()</code> (réseau, formule, contraintes, contrôles) sans lancer l'estimation.	Les appels générés sont valides pour différents cas d'usage et ne produisent pas d'erreur de traduction ou de paramétrage.	Validé sur cas tests (statique + longitudinal).	✓
↔	fit avec effets statiques	ERGM sur plusieurs partitions avec effets statiques uniquement via <code>erpm_long</code> .	Fit MCMLE OK, convergence raisonnable, stats cohérentes.	Validé.	✓
↔	fit avec effets inertiels	ERGM sur plusieurs partitions avec effets statiques + inertiels (via covariables d'historique).	Fit MCMLE OK, convergence raisonnable, stats cohérentes.	Dépend de l'existence d'un effet inertiel P2. Pas encore écrit.	△
effet inertiel	inertia_groups	Terme inertiel mesurant la persistance des groupes, défini par l'égalité exacte des ensembles d'acteurs entre partitions successives, sur une fenêtre temporelle donnée.	Sur des exemples contrôlés, la statistique varie conformément à la définition théorique.	Un seul effet inertiel en P2.	△
Transition MH		Ajout de nouvelles transitions MH (swap, split/merge) pour explorer l'espace des graphes plus efficacement.	Pour différentes configurations de mouvements, de pondérations et de termes, la chaîne MCMC explore l'espace des graphes conformément aux mouvements spécifiés et produit des modèles ERGM cohérents.	Dépend du format multi-toggle des change-stats.	△
↔	ergmterm format multi-toggle	Adaptation de l'ensemble des change-stats ERPM au format multi-toggle requis pour les nouvelles transitions MH.	Tous les termes ERPM supportent <code>ntoggles>1</code> sans incohérence.	Pré-requis pour l'implémentation de swap/merge/split.	△

↪	Fonctions C Steps	Implémentation des fonctions C générant des multi-toggles correspondant aux mouvements toggle, swap, merge et split.	Chaque fonction génère des multi-toggles valides et un log-ratio correct.	API interne, testable isolément.	△
↪	Proposal ErpmMix	Proposal C MH_ErpmMix : sélection aléatoire, à chaque itération MCMC, d'un type de mouvement selon des poids utilisateur, puis génération du multi-toggle correspondant.	Chaînes MCMC valides. Répartition des mouvements conforme aux pondérations.	Logratio 0 si symétrie garantie, sinon calculé.	△
↪	InitErgmProposal. ErpmMix	Écriture de InitProposal en R ErpmMix assurant la transmission des paramètres utilisateur vers le reste du code.	Appel depuis <code>control.ergm(MCMC.prop = ~ErpmMix(...))</code> fonctionnel.	Validation via dry-run + run court MCMC.	△
↪	<code>ergm()</code> : mh_moves / mh_weights	Extension de l'interface <code>ergm()</code> par des paramètres de contrôle des mouvements MCMC et traduction automatique vers la configuration <code>ergm</code> .	Interface utilisateur stable, contrôles d'inputs, valeurs par défaut raisonnables.	Point d'entrée utilisateur.	△

3 Synthèse étape par étape

1 ERPM long

Principe.

`erpm_long()` est conçu pour analyser une **série temporelle de partitions** $(P_1, \dots, P_T)$ en procédant de manière séquentielle. À chaque instant t , un **modèle ERGM statique** est ajusté sur la partition observée P_t .

Les dépendances au passé ne sont pas modélisées par une estimation conjointe sur toute la trajectoire. Elles sont introduites indirectement via des *effets inertiels* : des quantités calculées à partir des partitions passées $(P_{t-1}, \dots, P_{t-d})$, puis attachées au réseau courant comme **covariables exogènes** au temps t .

Ainsi, chaque modèle est estimé indépendamment, tout en étant conditionné par des informations explicites issues de l'historique.

Cadre et notations.

À chaque instant t , on observe une partition p_t de N acteurs. Cette partition est représentée par un réseau biparti acteurs-groupes

$$B_t = (A, G_t, E_t),$$

où chaque acteur $i \in A$ est relié à *son* groupe courant par une unique arête. La contrainte `~ bipart` garantit cette représentation (une appartenance par acteur).

Modèle pseudo-longitudinal.

Plutôt que de définir une loi jointe unique sur $(P_1, \dots, P_T)$, `erpm_long()` construit une suite de modèles statiques enrichis (E_t) . À chaque temps t , on estime un ERGM de type

$$\Pr_{E_t}(P_t = p_t \mid X_t) \propto \exp(\beta_t^\top s(p_t; X_t)),$$

où X_t désigne l'ensemble des covariables observées au temps t , incluant à la fois des attributs statiques associés à la partition courante et des variables d'historique construites à partir des partitions passées $(P_{t-1}, \dots, P_{t-d})$.

Historique explicite sous forme d'attributs réseau.

Les informations passées $(P_{t-1}, \dots, P_{t-d})$ sont **pré-calculées** puis attachées au réseau courant B_t sous forme d'attributs **exogènes** (fixes pendant l'estimation).

On appelle *profondeur* d'un terme inertiel le nombre de partitions passées dont il a besoin pour être évalué au temps t . Autrement dit, une profondeur d signifie que le terme exploite des informations issues des partitions $P_{t-1}, \dots, P_{t-d}$.

Si un terme inertiel requiert une profondeur d , alors au temps t on attache exactement d attributs au réseau courant, un par lag $\ell \in \{1, \dots, d\}$, suivant la convention :

$$\text{erpm_inertia_}\langle term \rangle_\text{lag}\langle \ell \rangle,$$

Le wrapper ne fait qu'attacher ces quantités au réseau de la partition courante, c'est la changestat qui les interprète.

Activation des effets inertiels.

Un terme inertiel nécessitant d partitions passées n'est actif qu'à partir de

$$t > d,$$

Cette règle garantit que le terme inertiel n'est évalué que lorsque toutes les partitions passées dont il dépend sont effectivement observées.

Architecture logicielle.

Le code repose sur trois niveaux d'encapsulation :

- `erpm_long()` : gère la boucle temporelle, l'historique, l'activation des inerties ;

- `erpm()` : Traduit les termes ERPM en termes ERGM ;
- `ergm()` : réalise l'estimation (MLE/MCMLE et MCMC) sur le réseau biparti construit à partir de la partition courante.

Les effets inertiels sont **déclarés dans un registre** et activés automatiquement selon leur profondeur.

Interprétation et limite.

Ce design produit une suite de modèles valides (E_t), mais **pas** une loi jointe unique sur $(P_1, \dots, P_T)$. En contrepartie, il permet de définir un score factorisé pour comparer des trajectoires :

$$\text{score}(Q_{1:T}) = \prod_{t=1}^T \Pr_{E_t}(Q_t \mid \hat{\theta}_t, \hat{\gamma}_t),$$

qui s'interprète comme une pseudo-vraisemblance longitudinale pratique pour l'évaluation relative de séquences.

Résumé.

`erpm_long` :

- un ERGM statique ajusté à chaque instant t ;
- des effets de persistance introduits via des attributs exogènes issus du passé ;
- une activation conditionnée à la disponibilité de l'historique requis ($t > d$) ;
- aucune modélisation jointe de la trajectoire $(P_1, \dots, P_T)$.

2 inertia_groups : persistance stricte de groupes entiers

Principe.

`inertia_groups` mesure, au temps t , combien de groupes de la partition courante réapparaissent à l'identique (mêmes membres) dans une fenêtre de partitions

passées. Optionnellement, on peut ne compter que certaines tailles de groupes via `size`. L'effet est **strict** : déplacer un seul acteur suffit à rendre le groupe « nouveau ».

Cadre et notations.

On observe une partition $\gamma_t : \{1, \dots, N\} \rightarrow G_t$. Pour un groupe $g \in G_t$,

$$A_t(g) = \{i : \gamma_t(i) = g\}, \quad n_t(g) = |A_t(g)|,$$

On définit la *signature* (composition) d'un groupe :

$$\sigma_t(g) = A_t(g) \subseteq \{1, \dots, N\},$$

Le paramètre `past_influence` = $L \geq 1$ fixe la profondeur mémoire. Le terme n'est activé qu'à partir de $t \geq L + 1$. Le filtre de tailles `size` définit un ensemble $S \subset \mathbb{N}$, avec la convention $\mathbf{1}[n \in S] = 1$ si $S = \emptyset$ (`size=NULL`).

Données passées attachées (vue code).

Au temps t , on attache une mémoire *fixe* construite à partir des partitions passées, par exemple sous forme d'un ensemble (ou hash-set) de signatures encodées de façon canonique :

$$\Sigma_t^{(L)} = \bigcup_{\ell=1}^L \Sigma_{t-\ell}, \quad \Sigma_{t-\ell} = \{\sigma_{t-\ell}(h) : h \in G_{t-\ell}\},$$

Pendant l'optimisation au temps t , $\Sigma_t^{(L)}$ ne change pas ; seule la partition courante varie.

Définition formelle.

$$T_{\text{inertia_groups}}(\gamma_t; S, L) = \sum_{g \in G_t} \mathbf{1}[n_t(g) \in S] \mathbf{1}[\sigma_t(g) \in \Sigma_t^{(L)}],$$

Change-stat locale.

Un mouvement élémentaire déplace un acteur i d'un groupe a vers un groupe b . Seuls a et b changent (taille et signature). En notant

$$c(g) = \mathbf{1}[n_t(g) \in S] \mathbf{1}[\sigma_t(g) \in \Sigma_t^{(L)}],$$

on obtient la variation :

$$\Delta T_{\text{inertia_groups}} = c'(b) - c(b) + c'(a) - c(a),$$

où $c'(\cdot)$ est calculé après mise à jour des tailles et signatures.

Interprétation.

Un coefficient positif favorise des partitions qui *réutilisent* des groupes déjà observés dans le passé (stabilité de groupes entiers). Un coefficient négatif pénalise

cette réutilisation et pousse vers des recombinaisons.

Appels `erpm_long()`.

```
1 # Fenêtre de mémoire L=2 (effet actif à partir de t >= 3)
2 erpm_long(parts ~ groups + inertia_groups(past_influence = 2))
3
4 # Ne compter que les groupes de taille 2 ou 3
5 erpm_long(parts ~ groups + inertia_groups(size = 2:3, past_
   influence = 2))
```